



دانشگاه صنعتی شریف
دانشکده مهندسی کامپیوتر

پروژه‌های درس معماری کامپیوتر

استاد:

دکتر حمید سربازی آزاد

تاریخ تحویل:

۱۴۰۵/۰۴/۳۱

فهرست پروژه‌ها

۲	دسته‌بندی پروژه‌ها و توضیحات کلی
۴	پروژه ۱
۸	پروژه ۲
۱۲	پروژه ۳
۱۶	پروژه ۴
۱۹	پروژه ۵
۲۳	پروژه ۶
۲۶	پروژه ۷

دسته‌بندی پروژه‌ها و توضیحات کلی

پروژه‌های این مجموعه به دو دسته کلی تقسیم می‌شوند. دسته نخست شامل پروژه‌های سطح پایین است که تمرکز اصلی آن‌ها بر طراحی و توسعه مستقیم اجزای پردازنده و منطق سخت‌افزاری قرار دارد. دسته دوم شامل پروژه‌های سطح بالا است که تحلیل و توسعه زیرسامانه حافظه در محیط شبیه‌سازی را بررسی می‌کند.

پروژه‌های سطح پایین

پروژه‌های ۱، ۲، ۳ و ۵ در دسته پروژه‌های سطح پایین قرار می‌گیرند. در این پروژه‌ها دانشجو به‌صورت مستقیم با ساختار داخلی پردازنده، خط لوله^۱، واحدهای پردازشی و منطق کنترلی درگیر می‌شود. هدف اصلی این پروژه‌ها، آشنایی عمیق با نحوه عملکرد پردازنده در سطح ریزمعماری^۲ و پیاده‌سازی عملی مفاهیمی مانند اجرای برداری^۳، پردازنده‌های چندصدوری^۴، مدیریت مخاطرات^۵ و پیش‌بینی انشعاب^۶ است. در این دسته از پروژه‌ها انتظار می‌رود دانشجو تغییرات لازم را در ساختار پردازنده، ماشین‌های حالت^۷، مسیر داده^۸ و واحد کنترل^۹ اعمال کرده و عملکرد صحیح سامانه را از طریق برنامه‌های آزمایشی و شبیه‌سازی واریسی کند.

پروژه‌های سطح بالا

پروژه‌های ۴، ۶ و ۷ در دسته پروژه‌های سطح بالا قرار می‌گیرند. تمرکز این پروژه‌ها بر زیرسامانه حافظه و تحلیل رفتار معماری در مقیاسی بزرگ‌تر است. در این پروژه‌ها، به‌جای طراحی مستقیم اجزای پردازنده، دانشجو با معماری حافظه‌های نهان، سیاست‌های مدیریت داده، سازوکارهای بهبود کارایی و تحلیل آماری عملکرد سامانه روبه‌رو می‌شود. پیاده‌سازی این پروژه‌ها در بستر شبیه‌ساز آکیتا^{۱۰} انجام می‌شود و هدف اصلی آن‌ها، تقویت توانایی دانشجو در تحلیل سامانه‌های پیچیده، بررسی رفتار حافظه و ارزیابی تأثیر تصمیمات معماری بر عملکرد نهایی سامانه است.

آشنایی با شبیه‌ساز آکیتا

آکیتا یک چارچوب شبیه‌سازی رویداد-محور^{۱۱} است که برای مدل‌سازی سامانه‌های رایانه‌ای پیچیده طراحی شده است. این چارچوب امکان توسعه و تحلیل اجزای مختلف معماری رایانه، به‌ویژه زیرسامانه حافظه و سامانه‌های چندپردازنده‌ای^{۱۲} را فراهم می‌کند.

در آکیتا، هر جزء معماری به‌صورت یک واحد مستقل مدل‌سازی می‌شود که از طریق ارسال و دریافت پیام با سایر

¹Pipeline

²Microarchitecture

³Vector Execution

⁴Superscalar Processors

⁵Hazard Management

⁶Branch Prediction

⁷State Machines

⁸Data Path

⁹Control Unit

¹⁰Akita

¹¹Event-Driven Simulation Framework

¹²Multiprocessor Systems

اجزا ارتباط برقرار می‌کند. حافظه‌های نهان، کنترل‌کننده‌های حافظه^{۱۳}، گذرگاه‌ها^{۱۴} و سایر اجزای سامانه به‌صورت پیمانه‌ای^{۱۵} پیاده‌سازی شده‌است و دانشجو می‌تواند با توسعه یا تغییر این اجزا، معماری‌های جدید را ارزیابی کند. در پروژه‌های سطح بالا، دانشجویان موظف هستند ضمن آشنایی با ساختار داخلی آکیتا، اجزای جدیدی به سامانه اضافه کرده یا منطق اجزای موجود را تغییر دهند و سپس تأثیر این تغییرات را با استفاده از معیارهای عملکردی مناسب مورد ارزیابی قرار دهند.

¹³Memory Controllers

¹⁴Buses

¹⁵Modular

پروژه ۱: طراحی سامانه پردازشی چندهسته‌ای و پیاده‌سازی قرارداد انسجام^{۱۶} حافظه

در این پروژه، قصد داریم با تکیه بر دانش طراحی پردازنده‌های تک‌هسته‌ای، به حوزه سامانه‌های چندپردازنده‌ای^{۱۷} وارد شویم. هدف اصلی این پروژه، ارتقای پردازنده‌ی مجهز به حافظه نهان و تبدیل آن به یک ساختار دوهسته‌ای است. چالش کلیدی در این مسیر، مدیریت حافظه‌های نهان اختصاصی و تضمین سازگاری داده‌ها^{۱۸} در زمان اجرای هم‌زمان است (در نظر داشته باشید ساختار پردازنده MIPS و رفتار دستورات جدید بر پایه معماری RISC-V RV32I است).

شرح ساختار کلی

در این معماری، هر دو هسته به یک حافظه اصلی مشترک متصل هستند، اما هر هسته برای افزایش سرعت دسترسی از یک حافظه نهان داده سطح اول^{۱۹} اختصاصی بهره می‌برد. استفاده از حافظه‌های نهان مجزا چالش بروز ناهمبستگی در داده‌های مشترک را ایجاد می‌کند، بنابراین، پیاده‌سازی روشی برای حفظ انسجام ضروری است. در این پروژه، قرارداد MESI، برای مدیریت وضعیت خطوط حافظه نهان و جلوگیری از خواندن داده‌های منقضی‌شده پیاده‌سازی می‌شود.

اهداف پروژه

- درک عملی چالش‌های اشتراک منابع در سامانه‌های چندهسته‌ای و تحلیل تأخیر دسترسی به حافظه،
- طراحی و پیاده‌سازی ماشین حالت قرارداد MESI و مدیریت انتقال وضعیت‌ها^{۲۰}،
- توسعه مجموعه دستورات با هدف پشتیبانی از شناسایی سخت‌افزاری هسته‌ها و عملیات اتمی^{۲۱}،
- تضمین صحت عملکرد سامانه در زمان دسترسی هم‌زمان به متغیرهای مشترک و نواحی بحرانی^{۲۲}.

توضیح فنی مشخصات حافظه نهان و قرارداد MESI

هر حافظه نهان در این پروژه دارای ساختار نگاشت مستقیم^{۲۳} با اندازه بلوک ۱۶ بایت است. سیاست نوشتار به صورت پس‌نویسی^{۲۴} همراه با تخصیص در زمان نوشتن^{۲۵} انتخاب شده است. برای مدیریت وضعیت هر خط، از چهار حالت اصلی استفاده می‌شود:

¹⁶Coherence

¹⁷Multiprocessor Systems

¹⁸Data Consistency

¹⁹L1 Data Cache

²⁰State Transitions

²¹Atomic Operations

²²Critical Sections

²³Direct-Mapped

²⁴Write-Back

²⁵Write-Allocate

- تغییر یافته (M): داده فقط در این حافظه نهان موجود است و با حافظه اصلی سازگار نیست.
- انحصاری (E): داده فقط در این حافظه نهان موجود است و با حافظه اصلی سازگار است.
- مشترک (S): داده در هر دو حافظه نهان موجود است و با حافظه اصلی همگام است.
- نامعتبر (I): خط حافظه نهان حاوی داده معتبر نیست.

ارتباط و هماهنگی بین دو هسته از طریق یک گذرگاه^{۲۶} مشترک و سیگنال‌های نظارتی نظیر BusRd و BusRdX انجام می‌گیرد.

مراحل پیاده‌سازی و نکات فنی

(برای انجام این بخش، می‌توانید پیاده‌سازی تمرین ۶ را مبنا قرار داده و آن را توسعه دهید.)

۱. توسعه مجموعه دستورات^{۲۷}

- دستور cpuid: این دستور شناسه هسته جاری (۰ یا ۱) را در ثبات مقصد قرار می‌دهد تا امکان تفکیک رفتار نرم‌افزار بر اساس هسته اجراکننده فراهم شود.
- دستورات اتمی (A-extension): پیاده‌سازی دستورات بارگذاری رزرو شده^{۲۸} و ذخیره‌سازی شرطی^{۲۹} به‌منظور فراهم کردن سازوکارهای همگام‌سازی و ایجاد قفل‌های نرم‌افزاری انجام می‌شود. همچنین، دستور amoad.w باید برای پشتیبانی از عملیات جمع اتمی در حافظه پیاده‌سازی شود تا خواندن، محاسبه و نوشتن مقدار به‌صورت یک عملیات تفکیک‌ناپذیر انجام گیرد.

۲. پیاده‌سازی کنترل‌کننده انسجام و گذرگاه

- طراحی منطق داوری گذرگاه^{۳۰} به روش نوبتی^{۳۱} برای مدیریت تراکنش‌های حافظه.
- پیاده‌سازی عملکرد تخلیه^{۳۲} برای بازگرداندن داده‌های تغییر یافته به حافظه اصلی در صورت نیاز هسته دیگر.
- مدیریت ابطال^{۳۳} خطوط حافظه نهان در زمان عملیات نوشتن توسط هسته دیگر.

²⁶Bus

²⁷ISA Extension

²⁸Load-Reserved (lr.w)

²⁹Store-Conditional (sc.w)

³⁰Bus Arbitration

³¹Round-Robin

³²Flush

³³Invalidation

۳. مدیریت رزرو و همگام‌سازی

- طراحی یک واحد ثبت رزرو^{۳۴} در هر هسته.
- پایش وضعیت آدرس‌های رزرو شده (اگر هسته دیگر روی آدرس هدف هسته جاری بنویسد، رزرو مذکور بلافاصله باطل می‌شود تا دستور SC.W شکست بخورد).

۴. تحلیل و سنجش عملکرد

- اجرای برنامه‌های موازی و نمایش صحت تبادل داده‌ها بین دو حافظه نهان.
- استخراج زمان دسترسی و تحلیل تأثیر برخوردها در گذرگاه بر کارایی کل سامانه.
- بررسی موازنه میان پیچیدگی قرارداد انسجام و صحت داده‌ها در شرایط رقابتی^{۳۵}.

۵. جزئیات فنی دستورات و تعامل با قرارداد انسجام

در این بخش، نحوه عملکرد دقیق دستورات افزوده شده و پیش‌نیازهای اجرای آن‌ها در سطح حافظه نهان شرح داده می‌شود. برای تضمین اتمی بودن، هر دستور باید گام‌های کنترلی مشخصی را در گذرگاه طی کند.

الف) دستورات اتمی:

- دستور (rs1), rd, lr.w: این دستور کلمه ۳۲ بیتی را از آدرس موجود در rs1 خوانده و در rd قرار می‌دهد. هم‌زمان، یک نشانگر رزرو برای این آدرس در هسته فعال می‌شود.
- دستور (rs1), rs2, rd, sc.w: تنها در صورتی که رزرو آدرس rs1 از زمان اجرای lr.w قبلی معتبر باقی مانده باشد، مقدار rs2 را در حافظه می‌نویسد و مقدار 0 را در rd برمی‌گرداند. در صورت ابطال رزرو توسط هسته دیگر، هیچ نوشتنی انجام نمی‌شود و مقدار 1، به‌عنوان نشانه شکست در rd درج می‌گردد.
- دستور (rs1), rs2, rd, amoadd.w: یک عملیات خواندن-اصلاح-نوشتن^{۳۶} را به‌صورت یکپارچه انجام می‌دهد. مقدار قبلی حافظه در rd ذخیره شده و حاصل جمع آن با rs2 دوباره در همان آدرس نوشته می‌شود.

ب) پیوند عملیاتی با قرارداد MESI:

برای اجرای صحیح و بدون نقص عملیات اتمی در سطح سامانه چندهسته‌ای، رعایت الزامات زیر در منطق کنترل‌کننده حافظه نهان ضروری است:

- انحصار پیش از نوشتن: خط حافظه نهان مربوط به آدرس هدف در دستورات اتمی، باید پیش از انجام عملیات نوشتن حتماً به وضعیت تغییریافته منتقل شود.
- مدیریت مالکیت: در صورتی که خط در وضعیت مشترک (S) باشد، هسته باید با ارسال سیگنال BusRdX در گذرگاه، مالکیت انحصاری خط را به دست آورده و نسخه‌های موجود در هسته دیگر را ابطال کند.

³⁴Reservation Station

³⁵Race Condition

³⁶Read-Modify-Write

- ابطال رزرو: اگر هسته مجاور، برای مثال هسته ۱، روی خطی بنویسد که توسط هسته جاری، برای مثال هسته ۰، رزرو شده است، منطق نظارتی^{۳۷} باید بلافاصله بیت رزرو هسته ۰ را باطل نماید. این فرآیند پایه اصلی پیاده‌سازی همگام‌سازی بدون قفل^{۳۸} در معماری RISC-V است.

خروجی‌های مورد انتظار در گزارش و کدهای تحویلی

گزارش نهایی پروژه باید شامل مستندات زیر باشد:

- رسم نمودار^{۳۹} دقیق معماری پردازنده دوهسته‌ای و نحوه اتصال حافظه‌ها به گذرگاه مشترک،
- تشریح کامل ماشین حالت قرارداد MESI و واکنش‌های آن به درخواست‌های خواندن/نوشتن محلی و سیگنال‌های گذرگاه،
- توضیحات فنی درباره نحوه اضافه کردن دستورات جدید به خط لوله و مدیریت وابستگی‌های داده‌ای در دستورات اتمی،
- نمودارهای زمانی^{۴۰} که فرآیند جابه‌جایی مالکیت داده بین هسته‌ها را نشان دهد.

علاوه بر گزارش، موارد زیر باید تحویل داده شود:

- کد پیاده‌سازی شده پردازنده، به زبان توصیف سخت‌افزار یا محیط شبیه‌سازی مربوطه،
- نتایج اجرای برنامه‌های آزمون که نشان‌دهنده عملکرد صحیح سامانه در سناریوهای اشتراک داده است،
- تحلیل خروجی‌های مربوط به وضعیت حافظه نهان، شامل تعداد دفعات اصابت و تغییر وضعیت‌های قرارداد.

³⁷Snooping Logic

³⁸Lock-Free Synchronization

³⁹Diagram

⁴⁰Timing Diagrams

پروژه ۲: طراحی معماری و پیاده‌سازی واحد محاسبات برداری (VPU)^{۴۱}

در این پروژه، قصد داریم به یکی از راهکارهای اساسی برای افزایش توان پردازشی، یعنی اضافه کردن یک واحد محاسبات برداری به پردازنده مبتنی بر معماری RISC-V RV32I، بپردازیم (در نظر داشته باشید ساختار پردازنده همانند تمرین‌ها MIPS و رفتار دستورات جدید بر پایه معماری RISC-V RV32I است). این ارتقا به‌ویژه در برنامه‌هایی که نیازمند پردازش حجم انبوهی از داده‌ها هستند، کاربرد فراوان دارد.

شرح مسئله و ساختار کلی

معمولاً در پردازنده‌های استاندارد و سنتی، اجرای محاسبات روی آرایه‌های بزرگ نیازمند اجرای حلقه‌های طولانی است. در این حالت، پردازنده مجبور است برای هر عنصر داده، دستورات تکراری را واکنشی و اجرا کند که این امر جریمه زمانی سنگین و مصرف منابع بالایی را به همراه دارد.

برای حل این مشکل، واحد برداری به‌صورت یک پیمانه سخت‌افزاری مستقل در کنار پردازنده اصلی^{۴۲} قرار می‌گیرد. این واحد از طریق یک کانال ارتباطی مشخص با پردازنده تعامل دارد و شامل بانک ثبات‌های برداری، واحد محاسبات برداری (Vector ALU / FPU)^{۴۳} و یک کنترل‌کننده حافظه مستقل است. با این معماری، VPU قادر است یک دستور واحد را دریافت کرده و آن را به‌صورت موازی روی مجموعه‌ای از داده‌ها اعمال کند. در این طراحی، حافظه اصلی^{۴۴} و حافظه نهان^{۴۵} بین CPU و VPU به‌صورت مشترک استفاده می‌شوند.

اهداف پروژه

- درک عملی معماری‌های برداری^{۴۶} و بهره‌گیری از قابلیت موازی‌سازی در سطح داده^{۴۷}،
- تحلیل مزایای اجرای محاسبات تکرارشونده و داده‌محور به‌صورت موازی در مقایسه با روش اسکالر^{۴۸}،
- طراحی و پیاده‌سازی سخت‌افزاری بانک ثبات‌های مستقل و کنترل‌کننده حافظه برداری،
- مدیریت فرایند نسبتاً پیچیده همگام‌سازی و تبادل وضعیت بین خط لوله پردازنده اصلی و پیمانه برداری.

توضیح مفهومی عملکرد اجرای برداری^{۴۹}

در معماری‌های پایه اسکالر، برای جمع دو آرایه، پردازنده باید درگیر یک چرخه تکراری از واکنشی دستور، خواندن عملوندها و نوشتن نتایج شود. اما با اضافه شدن VPU، پردازنده اصلی تنها یک دستور برداری و آدرس‌های اولیه را صادر می‌کند.

در این فرایند، دستور برداری از CPU به VPU ارسال می‌شود و با فعال شدن سیگنال شروع^{۵۰}، واحد برداری

⁴¹Vector Processing Unit - VPU

⁴²(CPU)

⁴³Floating-Point Unit - FPU

⁴⁴Main Memory

⁴⁵Cache

⁴⁶Vector Architectures

⁴⁷Data-Level Parallelism - DLP

⁴⁸Scalar

⁴⁹Vector Execution

⁵⁰Start

وارد حالت مشغول^{۵۱} می‌شود. سپس داده‌ها را به صورت مستقل از حافظه واکشی کرده و عملیات را به شکل موازی یا خط لوله‌ای^{۵۲} روی عناصر اجرا می‌کند. پس از پایان کار، سیگنال اتمام^{۵۳} برای پردازنده فعال می‌شود. این روش با کاهش واکشی و اجرای دستورات تکراری، باعث افزایش سرعت اجرای برنامه می‌شود.

مراحل پیاده‌سازی و نکات فنی

(برای انجام این بخش، می‌توانید پیاده‌سازی تمرین ۶ یا ۷ را مبنا قرار داده و آن را توسعه دهید.)

۱. طراحی بانک ثبات‌های برداری مستقل^{۵۴}

- در این بخش، باید یک مجموعه اختصاصی شامل حداقل ۸ ثبات برداری طراحی و پیاده‌سازی شود،
- هر ثبات باید شامل چندین عنصر 32 بیتی باشد و طول بردار^{۵۵} باید در سامانه قابل تنظیم باشد،
- دسترسی به این ثبات‌ها منحصراً در اختیار VPU است. CPU اجازه دسترسی مستقیم به این ثبات‌ها را ندارد و انتقال داده بین پردازنده و ثبات‌های برداری صرفاً باید از طریق رابط مشخص شده انجام پذیرد.

۲. طراحی قراردادهای ارتباطی و همگام‌سازی^{۵۶}

- ارسال پیام‌ها: ایجاد یک کانال سخت‌افزاری برای ارسال دستور برداری، طول بردار، آدرس حافظه برای بارگذاری و ذخیره‌سازی^{۵۷} و سیگنال شروع اجرا.
- دریافت وضعیت: مدیریت سیگنال‌های بازگشتی شامل وضعیت مشغول، آماده و اتمام عملیات.
- کنترل تداخل: پردازنده اصلی باید به گونه‌ای طراحی شود که در زمان مشغول بودن VPU، از صدور دستور برداری جدید خودداری کند.

۳. پیاده‌سازی کنترل‌کننده حافظه مستقل^{۵۸}

- طراحی پیمانه‌ای درون VPU که به صورت مستقل درخواست‌های متوالی حافظه را بدون دخالت مستقیم پردازنده تولید کند.
- پشتیبانی از عملیات بارگذاری برداری (یعنی خواندن متوالی عناصر از حافظه و قرار دادن آن‌ها در ثبات برداری) و ذخیره‌سازی برداری (یعنی نوشتن متوالی عناصر در حافظه).

⁵¹Busy

⁵²Pipelined

⁵³Done

⁵⁴Vector Register File

⁵⁵Vector Length

⁵⁶Synchronization

⁵⁷Load/Store

⁵⁸Independent Memory Controller

- مدیریت هوشمندانه دسترسی به حافظه مشترک برای جلوگیری از بن‌بست^{۵۹} یا تداخل با درخواست‌های CPU.

۴. پیاده‌سازی مجموعه دستورات برداری^{۶۰}

شما باید یک مجموعه کاربردی اما حداقلی از دستورات RISC-V را برای این واحد طراحی کنید:

- جمع برداری اعداد صحیح (vadd.vv): محاسبه $vd[i] = vs1[i] + vs2[i]$ برای تک‌تک عناصر.
- ضرب برداری اعداد صحیح (vmul.vv): محاسبه $vd[i] = vs1[i] \times vs2[i]$
- محاسبات ممیز شناور^{۶۱}: پشتیبانی از حداقل یک دستور مانند جمع (vfadd.vv) یا ضرب.
- انتقال بلوکی داده: دستورات واکشی (rs1) vload vd, (rs1) و ذخیره‌سازی (rs1) vstore vs1.

۵. همگام‌سازی پیشرفته خط لوله^{۶۲} (بخش اختیاری)

برای دستیابی به یک سامانه کارآمدتر و جلوگیری از نقصان‌های احتمالی در زمان اجرای موازی، می‌توانید عملکردهای همگام‌سازی زیر را بین خط لوله CPU و VPU پیاده‌سازی کنید:

- کنترل صدور دستورات برداری^{۶۳}: پردازنده اصلی باید به‌گونه‌ای هوشمند عمل کند که در صورت فعال بودن سیگنال مشغول^{۶۴} از سوی واحد برداری، از صدور هرگونه دستور برداری جدید^{۶۵} خودداری نماید. در این حالت، خط لوله CPU در مرحله مربوطه متوقف^{۶۶} می‌شود تا زمانی که VPU دوباره به حالت آماده^{۶۷} بازگردد.
- پایش لحظه‌ای وضعیت^{۶۸}: سیگنال‌های وضعیت (Busy/Ready) باید به‌صورت ترکیبی یا در هر چرخه ساعت^{۶۹} در دسترس واحد کنترل CPU باشند. این امر باعث می‌شود که پردازنده بدون تأخیر اضافی، بلافاصله پس از آزاد شدن واحد برداری، دستور بعدی را صادر کند.
- مدیریت تداخل دسترسی به حافظه^{۷۰}: از آنجایی که هر دو واحد از حافظه مشترک استفاده می‌کنند، در صورتی که یک دستور برداری در حال انتقال داده بین حافظه و بانک ثبات برداری باشد، باید بخشی برای اولویت‌بندی یا نوبت‌دهی تعبیه شود. این قرارداد باید تضمین کند که دسترسی‌های هم‌زمان به گذرگاه حافظه^{۷۱} باعث خرابی یا ناسازگاری داده‌ها و همچنین بن‌بست در سامانه نمی‌شود.

⁵⁹Deadlock

⁶⁰Vector Instruction Set

⁶¹Floating Point

⁶²Advanced Pipeline Synchronization

⁶³Instruction Issue Control

⁶⁴Busy

⁶⁵(Issue)

⁶⁶Stall

⁶⁷Ready

⁶⁸Real-Time Status Monitoring

⁶⁹Clock Cycle

⁷⁰Memory Access Conflict Management

⁷¹Memory Bus

۶. مقایسه قبل و بعد از استفاده از واحد برداری

- انتخاب و اجرای یک برنامه محک^{۷۲} داده‌محور مانند جمع دو آرایه، ضرب برداری، محاسبه میانگین یک آرایه یا یک فیلتر ساده عددی،
- اجرای برنامه یک بار به صورت کاملاً اسکالر، روی CPU و بار دیگر به صورت برداری، توسط VPU،
- استخراج زمان کل اجرا^{۷۳}، تعداد دستورات پردازش شده و تعداد دفعات دسترسی به حافظه در هر دو حالت.
- محاسبه و مقایسه سرعت نسبی سامانه با استفاده از فرمول:

$$Speedup = \frac{Execution\ Cycles_{Scalar}}{Execution\ Cycles_{Vector}}$$

خروجی‌های مورد انتظار در گزارش و کدهای تحویلی

گزارش نهایی شما باید شامل موارد زیر باشد:

- رسم نمودار دقیق معماری سامانه (CPU-VPU) و نمودار ساختار داخلی بانک ثبات‌های برداری،
- توضیحات جامع درباره نحوه مدل‌سازی رابط سخت‌افزاری و ماشین حالت^{۷۴} مربوط به همگام‌سازی خط لوله‌ها،
- شرح گام‌به‌گام نحوه اجرای یک دستور برداری در سخت‌افزار و نحوه تولید خودکار آدرس‌های حافظه،
- تحلیل ریاضی و مقایسه‌ای عملکرد سامانه در دو حالت اسکالر و برداری،
- جداول و نمودارهای مقایسه‌ای که میزان کاهش چرخه‌های اجرا و کاهش بار ترافیک دستورات را به وضوح نشان دهند.

علاوه بر گزارش، نتایج آزمون شما نیز باید شامل این موارد باشد:

- فایل‌های کد سخت‌افزاری نوشته‌شده، شامل توصیف پردازنده و واحد برداری، به همراه فایل‌های شبیه‌سازی تست‌بنچ^{۷۵}،
- ارائه مشاهدات عملی از اجرای برنامه‌ها و نمایش تغییرات سیگنال‌های کنترلی (Busy, Ready, Start) در طول زمان شبیه‌سازی،
- تحلیل کمی بهره‌وری^{۷۶} پردازش موازی و بررسی محدودیت‌ها یا گلوگاه‌های^{۷۷} سامانه در هنگام دسترسی مشترک به حافظه.

⁷²Benchmark

⁷³Execution Time

⁷⁴State Machine

⁷⁵Testbench

⁷⁶Efficiency

⁷⁷Bottlenecks

پروژه ۳: طراحی معماری و پیاده‌سازی پردازنده سوپراسکالر^{۷۸}

در این پروژه، قصد داریم به یکی از راهکارهای پیشرفته در معماری کامپیوتر برای افزایش توان پردازشی بپردازیم. شما پردازنده خط لوله‌ای^{۷۹} پنج‌مرحله‌ای را در تمرین‌های قبلی طراحی کرده‌اید و اکنون آن را به یک پردازنده سوپراسکالر دوصدوری^{۸۰} تبدیل خواهید کرد (در نظر داشته باشید ساختار پردازنده همانند تمرین‌ها MIPS و رفتار دستورات جدید بر پایه معماری RISC-V RV32I است). در این معماری پیشرفته، به جای وجود یک خط لوله واحد، پردازنده دارای دو خط لوله موازی و هم‌زمان خواهد بود که می‌توانند در هر چرخه ساعت، حداکثر دو دستور مستقل را صادر و اجرا کنند.

شرح مسئله و ساختار کلی

در پردازنده‌های خط لوله‌ای استاندارد، محدودیت ذاتی صدور یک دستور در هر چرخه، مانع از رسیدن شاخص IPC^{۸۱} به اعداد بالاتر از یک می‌شود. برای غلبه بر این محدودیت، معماری سوپراسکالر این امکان را فراهم می‌کند که در صورت عدم وجود وابستگی بین دستورات، چندین دستور به صورت هم‌زمان وارد مرحله اجرا شوند. سامانه نهایی شما شامل عناصر سخت‌افزاری زیر خواهد بود:

- دو خط لوله پردازشی موازی: هر خط لوله شامل مراحل پنج‌گانه استاندارد باشد (IF, ID, EX, MEM, WB).
- واحد واکنشی دستور^{۸۲} مشترک: با قابلیت واکنشی دو دستور متوالی در هر چرخه،
- واحد رمزگشایی^{۸۳} مشترک: با توانایی بررسی و تحلیل موازی دو دستور،
- بانک ثبات‌های مشترک^{۸۴}: به صورت هم‌زمان به هر دو خط لوله سرویس می‌دهد،
- حافظه اصلی مشترک^{۸۵}.
- منطق کنترلی صدور دوگانه^{۸۶}: بخش اصلی تصمیم‌گیری پردازنده برای ارسال دستورات به خطوط لوله.

اهداف پروژه

- درک عملی مفهوم موازی‌سازی در سطح دستورالعمل^{۸۷} و نحوه استخراج آن توسط سخت‌افزار،
- طراحی و مدل‌سازی منطق پیچیده صدور هم‌زمان دو دستور،

⁷⁸Superscalar Processor

⁷⁹Pipelined Processor

⁸⁰Dual-Issue Mini Superscalar

⁸¹Instructions Per Cycle

⁸²Instruction Fetch - IF

⁸³Instruction Decode - ID

⁸⁴Shared Register File

⁸⁵RAM

⁸⁶Dual-Issue Control Logic

⁸⁷Instruction-Level Parallelism - ILP

- شناسایی و مدیریت صحیح مخاطرات^{۸۸} در یک پردازنده چندصدوری،
- تحلیل تأثیر افزایش پهنای باند پردازنده بر کاهش تعداد چرخه‌های اجرا.

توضیح مفهومی منطق صدور دوگانه

قلب این معماری، منطق صدور دستورات است. این واحد در هر چرخه ساعت باید فرایند زیر را طی کند:

۱. دو دستور متوالی از حافظه برنامه واکنشی شود،
 ۲. ارزیابی دقیق در مرحله رمزگشایی (ID) تا مشخص شود آیا این دو دستور می‌توانند در کنار هم صادر شوند یا خیر،
 ۳. در صورت کشف هرگونه وابستگی یا تداخل منابع، سخت‌افزار تنها دستور اول، یعنی دستور قدیمی‌تر، را صادر کرده و دستور دوم را متوقف (Stall) می‌کند تا در چرخه بعدی دوباره بررسی شود.
- شرایط الزامی برای صدور همزمان دو دستور:
- عدم وجود وابستگی داده‌ای خواندن پس از نوشتن (RAW) بین دستور اول و دوم،
 - عدم وجود تداخل در ثبات مقصد یا مخاطره نوشتن پس از نوشتن (WAW)،
 - عدم رقابت بر سر استفاده از منابع انحصاری، مانند تلاش همزمان هر دو دستور برای دسترسی به یک پورت واحد از حافظه RAM.

مراحل پیاده‌سازی و نکات فنی

(برای انجام این بخش، می‌توانید پیاده‌سازی تمرین ۷ را مبنا قرار داده و آن را توسعه دهید.)

۱. حل مخاطرات داده‌ای^{۸۹}

- مخاطره RAW، خواندن پس از نوشتن: اگر دستور دوم به نتیجه محاسباتی دستور اول وابسته باشد، سه راهکار پیش رو دارید: جلوگیری از صدور همزمان، استفاده از عملکرد هدایت داده^{۹۰}، یا توقف کامل خط لوله. پیاده‌سازی حداقلی و قابل قبول: جلوگیری از صدور همزمان دو دستور وابسته، یعنی توقف دستور دوم،
- مخاطره WAW، نوشتن پس از نوشتن: اگر هر دو دستور قصد نوشتن داده روی یک ثبات یکسان را داشته باشند، صدور همزمان مجاز نیست و یکی از دستورات، یعنی دستور جدیدتر، باید متوقف شود تا مقدار نهایی به‌درستی در ثبات قرار گیرد.

⁸⁸Hazards

⁸⁹Data Hazards

⁹⁰Forwarding

۲. حل مخاطرات کنترلی^{۹۱}

در زمان برخورد با دستورات پرش شرطی^{۹۲} یا پرش بدون شرط^{۹۳}، پردازنده باید از اجرای اشتباه دستورات پس از پرش جلوگیری کند. روش‌های مجاز شامل توقف خط لوله، پیش‌بینی ثابت، یا پاکسازی^{۹۴} دستورات است. پیاده‌سازی حداقلی و قابل قبول: استفاده از فرض ثابت عدم وقوع پرش (Always Not Taken) و دستورات نادرست از خط لوله در صورت تشخیص اشتباه در مسیر اجرای برنامه پاکسازی شوند.

۳. مدیریت دسترسی به حافظه

از آنجا که حافظه اصلی بین دو خط لوله مشترک است، اگر هر دو خط لوله در یک چرخه مشخص به مرحله MEM برسند:

- تنها یک دسترسی، خواندن یا نوشتن، در هر چرخه مجاز است،
- در صورت هم‌زمانی درخواست‌ها، یکی از خطوط لوله باید متوقف (Stall) شود،
- شما باید یک سیاست انتخاب شفاف طراحی کنید. به‌عنوان مثال، اختصاص اولویت همیشگی به خط لوله شماره ۰. دقت کنید که تحت هیچ شرایطی نباید تداخل ناسازگار در دسترسی به RAM رخ دهد.

۴. پیاده‌سازی ساختار فایل ثبات‌ها

- این پیمان‌ها باید قادر باشد پورت‌های خواندن کافی برای تأمین هم‌زمان عملوندهای هر دو دستور را فراهم کند،
- باید امکان دو عملیات نوشتن هم‌زمان، در صورت متفاوت بودن آدرس ثبات‌ها، فراهم شود،
- در صورت تداخل و تلاش برای نوشتن هم‌زمان روی یک ثبات مشترک، منطق کنترلی باید تنها یک عملیات نوشتن را معتبر بداند.

۵. آزمون و ارزیابی عملکرد

شما موظف هستید حداقل دو نوع برنامه محک^{۹۵} متمایز را روی پردازنده خود اجرا کنید:

۱. برنامه با دستورات کاملاً مستقل: مانند توالی دستورات (add x1, x2, x3)، (add x4, x5, x6) و (add x7, x8, x9). در این حالت ایده‌آل، پردازنده باید بتواند در بیشتر چرخه‌ها دو دستور را به‌طور هم‌زمان صادر کند،

۲. برنامه با دستورات وابسته: مانند توالی دستورات (add x1, x2, x3)، (add x4, x1, x5) و (add x6, x4, x7). در این حالت، وابستگی‌های متوالی باید منجر به تولید سیگنال‌های توقف (Stall) شده و عملکرد صحیح سامانه را به چالش بکشد.

^{۹۱}Control Hazards

^{۹۲}Branch

^{۹۳}Jump

^{۹۴}Flush

^{۹۵}Benchmark

سپس باید پارامترهای زیر را استخراج و مقایسه کنید:

- تعداد چرخه‌های کل اجرا در حالت تک خط لوله‌ای، یعنی معماری پایه،
- تعداد چرخه‌های کل اجرا در حالت دو خط لوله‌ای، یعنی معماری سوپراسکالر،
- محاسبه شاخص $CPI^{۹۶}$ مؤثر سامانه در هر دو حالت.
- محاسبه درصد بهبود عملکرد و ارزیابی میزان افزایش پهنای باند پردازنده.

خروجی‌های مورد انتظار در گزارش و کدهای تحویلی

گزارش نهایی شما باید شامل موارد زیر باشد:

- رسم نمودار^{۹۷} دقیق معماری پردازنده Mini SuperScalar شامل تمامی خطوط ارتباطی و پیمانه‌های مشترک،
 - توضیحات جامع درباره نحوه عملکرد منطق صدور دو دستور و ماشین حالت^{۹۸} مربوط به آن،
 - تشریح کامل راهبردهای به‌کاررفته برای مدیریت تمامی مخاطرات داده‌ای (RAW، WAW) و مخاطرات کنترلی،
 - توضیح نحوه مدیریت تداخل در پورت‌های حافظه در صورت رسیدن هم‌زمان دو دستور،
 - تحلیل مقایسه‌ای عملکرد پردازنده پایه نسبت به پردازنده ارتقایافته و تحلیل افزایش پهنای باند پردازنده.
- علاوه بر گزارش، کدهای تحویلی و نتایج آزمون شما نیز باید شامل این موارد باشد:
- فایل‌های کد منبع سخت‌افزاری، شامل محیط شبیه‌سازی و توصیف پردازنده،
 - ارائه مشاهدات عملی از اجرای برنامه‌های مستقل و وابسته به کمک شکل‌موج‌ها^{۹۹}،
 - ارائه جدول مقایسه چرخه‌ها و اثبات کمی بهبود کارایی معماری پیاده‌سازی‌شده.

^{۹۶}Cycles Per Instruction - CPI

^{۹۷}Diagram

^{۹۸}State Machine

^{۹۹}Waveforms

پروژه ۴: واحد پیش‌واکشی حافظه^{۱۰۰}

در طراحی معماری پردازنده‌ها، شکاف سرعت میان پردازنده و حافظه اصلی یک چالش اساسی است. هنگام بروز نقصان عدم اصابت^{۱۰۱}، پردازنده تا زمان انتقال داده از حافظه متوقف^{۱۰۲} می‌شود که افت شدید کارایی را در پی دارد. برای پنهان‌سازی این تأخیر، تکنیک پیش‌واکشی در این پروژه مورد استفاده قرار می‌گیرد. واحد پیش‌واکشی با بررسی جریان درخواست‌های خواندن حافظه^{۱۰۳}، رفتار آینده برنامه را پیش‌بینی می‌کند. در صورت پیش‌بینی صحیح، داده‌ها پیش از درخواست صریح پردازنده، در پس‌زمینه به حافظه نهان سطح دوم (*L2 Cache*) منتقل می‌گردند. در مقابل، پیش‌بینی نادرست موجب آلودگی حافظه نهان^{۱۰۴} و هدررفت پهنای باند گذرگاه می‌شود. پیاده‌سازی این عملکرد نیازمند درک دقیق فرایند ترجمه آدرس‌های مجازی به فیزیکی است.

شرح مسئله و ساختار کلی

در این پروژه قصد داریم یک واحد پیش‌واکشی گام‌دار^{۱۰۵} به حافظه نهان در شبیه‌ساز اضافه کنیم که از طریق پارامترهای برنامه قابل تنظیم باشد. کاری که لازم است انجام شود، ایجاد یک عنصر جدید در شبیه‌ساز و اضافه کردن آن به حافظه نهان است؛ این عنصر باید درخواست‌های ورودی را پایش کرده و عملیات پیش‌واکشی را انجام دهد. توجه داشته باشید که پیش‌واکشی تنها در حالتی انجام می‌شود که صفحه موردنظر در حافظه نهان موجود نباشد. پس از پیاده‌سازی این واحد، لازم است تحلیل‌های آماری روی ردهای حافظه^{۱۰۶} انجام دهید تا تأثیر این واحد را نمایش دهید. این بخش را می‌توانید با هر زبان برنامه‌سازی دلخواه انجام دهید.

اهداف پروژه

- پیاده‌سازی سخت‌افزاری عملکردهای پنهان‌سازی تأخیر حافظه و بررسی اثر آن‌ها بر زمان اجرای برنامه‌ها،
- شناخت عمیق مفاهیم مدیریت حافظه، ترجمه آدرس‌ها و عملکرد میانگیر ترجمه آدرس^{۱۰۷}،
- طراحی منطق پیش‌بینی برای دو الگوریتم پیش‌واکشی متفاوت،
- ارزیابی موازنه میان افزایش نرخ اصابت^{۱۰۸} و سربار ترافیک اضافی حافظه.

مفاهیم پایه ترجمه آدرس و مدیریت حافظه

- مبدل آدرس^{۱۰۹}: واحدی در سامانه حافظه است که وظیفه تبدیل آدرس‌های مجازی تولیدشده توسط پردازنده به آدرس‌های فیزیکی حافظه را بر عهده دارد. این تبدیل معمولاً توسط واحد مدیریت حافظه^{۱۱۰} انجام می‌شود

¹⁰⁰Memory Prefetch Unit

¹⁰¹Cache Miss

¹⁰²Stall

¹⁰³Memory Read Requests

¹⁰⁴Cache Pollution

¹⁰⁵Stride Prefetching Unit

¹⁰⁶Memory Traces

¹⁰⁷Translation Lookaside Buffer - TLB

¹⁰⁸Hit Rate

¹⁰⁹Address Translator

¹¹⁰Memory Management Unit - MMU

و شامل بررسی ساختارهایی مانند جدول صفحات^{۱۱۱} و میانگیر ترجمه آدرس است،

- میانگیر ترجمه آدرس (*TLB*): یک حافظه نهان کوچک و بسیار سریع است که نگاشتهای اخیر بین آدرس مجازی و آدرس فیزیکی را نگهداری می‌کند. هدف *TLB* کاهش زمان تبدیل آدرس است؛ در صورت وجود نگاشت در *TLB*، نیازی به مراجعه به جدول صفحات در حافظه اصلی نیست و دسترسی به حافظه سریع‌تر انجام می‌شود،

- آدرس مجازی^{۱۱۲}: آدرسی است که توسط برنامه و پردازنده تولید می‌شود و مستقل از محل واقعی داده در حافظه فیزیکی است. استفاده از آدرس مجازی امکان جداسازی فضای حافظه پردازش‌ها، افزایش امنیت و استفاده انعطاف‌پذیر از حافظه را فراهم می‌کند،

- آدرس فیزیکی^{۱۱۳}: آدرس واقعی محل ذخیره داده در حافظه اصلی سامانه است. پس از تبدیل آدرس مجازی توسط واحد مدیریت حافظه، دسترسی نهایی به داده با استفاده از این آدرس انجام می‌شود.

مراحل پیاده‌سازی و نکات فنی

۱. طراحی رابط پیش‌واکشی در حافظه نهان^{۱۱۴}:

باید یک رابط در عنصر حافظه نهان ایجاد شود که امکان فعال‌سازی واحد پیش‌واکشی را فراهم کند.

۲. افزودن پرچم کنترلی برای فعال‌سازی پیش‌واکشی‌کننده^{۱۱۵}:

باید امکان فعال‌سازی واحد پیش‌واکشی در حافظه نهان سطح دوم از طریق یک پرچم کنترلی فراهم شود. انتخاب نوع الگوریتم پیش‌واکشی نیز باید از طریق یک پارامتر ورودی امکان‌پذیر باشد.

۳. پیاده‌سازی پیش‌واکشی‌کننده گام‌دار:

این الگوریتم با تشخیص الگوی دسترسی با فاصله ثابت، تلاش می‌کند درخواست بعدی را پیش‌واکشی کند. در صورت تشخیص نقصان، پیش‌واکشی متوقف می‌شود تا الگوی جدید مشخص گردد.

۴. قابلیت تنظیم پارامترهای الگوریتم‌ها:

الگوریتم‌های پیش‌واکشی باید دارای پارامترهای قابل تنظیم باشند تا امکان تحلیل رفتار آن‌ها در شرایط مختلف سامانه فراهم شود.

¹¹¹Page Table

¹¹²Virtual Address

¹¹³Physical Address

¹¹⁴Prefetching Interface

¹¹⁵Prefetcher

۵. ارزیابی عملکرد پیش‌واکشی:

بررسی تأثیر هر الگوریتم بر عملکرد سامانه با استفاده از اطلاعات حافظه، شامل نرخ اصابت و نقصان^{۱۱۶} و کاهش میزان تأخیر دسترسی، الزامی است.

خروجی‌های مورد انتظار در گزارش و کدهای تحویلی

گزارش نهایی شما باید شامل موارد زیر باشد:

- ارائه توضیحی جامع و درعین حال خلاصه درباره الگوریتم پیاده‌سازی شده،
 - اثبات درستی عملکرد پیش‌واکشی با استفاده از گزارش اجرای برنامه^{۱۱۷}،
 - رسم نمودار و مقایسه عملکرد واحد پیش‌واکشی،
 - گزارش معیارهای مربوط به حافظه نهان. این معیارها شامل نرخ نقصان^{۱۱۸}، نرخ اصابت^{۱۱۹}، نرخ اصابت MSHR^{۱۲۰}، نرخ نقصان MSHR^{۱۲۱}، میزان پوشش^{۱۲۲}، زمان اجرا، درصد استفاده از پیش‌واکشی‌ها، تعداد بلاک‌های جایگزین شده به دلیل پیش‌واکشی، و تعداد نقصان‌هایی هستند که به همین دلیل رخ داده‌اند. توجه داشته باشید که اگر در برنامه محک شما مقدار یکی از این معیارها صفر بود، باید برنامه محک یا پیکربندی اجرای خود را تغییر دهید.
 - تحلیل گزارش ارائه شده و بررسی رفتار برنامه‌های محک با استفاده از اطلاعات استخراج شده.
- علاوه بر گزارش، لازم است کد شبیه‌ساز را به صورت شخصی در مخزن گیت‌هاب خود قرار داده و تغییرات را روی آن اعمال کنید. در زمان تحویل، از شما خواسته خواهد شد دسترسی‌های لازم برای مشاهده و بررسی مخزن را فراهم کنید. همچنین، فایل‌های کلیدی کد باید در کنار گزارش نهایی قرار داده شوند.
- MSHR^{۱۲۳} یک قطعه سخت‌افزاری است که وظیفه مدیریت و یکسان‌سازی درخواست‌های حافظه را بر عهده دارد. این عنصر دو وظیفه اصلی دارد: نخست، درخواست‌های حافظه مربوط به یک بلاک یکسان را جمع‌آوری کرده و درخواست‌های تکراری را حذف می‌کند تا از پردازش چندباره آن‌ها جلوگیری شود. دوم، اگر در زمان پاسخ‌گویی به یک نقصان Miss، درخواست دیگری برای همان بلاک ثبت شود، آن درخواست به عنوان اصابت در MSHR Hit شناخته می‌شود و تا زمان آماده شدن نتیجه، در همان بخش منتظر می‌ماند.

¹¹⁶Hit/Miss Rate

¹¹⁷Execution Log

¹¹⁸Miss Rate

¹¹⁹Hit Rate

¹²⁰MSHR Hit Rate

¹²¹MSHR Miss Rate

¹²²Coverage

¹²³Miss Status Holding Register

پروژه ۵: تحلیل مخاطرات کنترلی و طراحی معماری پیش‌بینی انشعاب^{۱۲۴} در پردازنده خط لوله

در این پروژه، قصد داریم به یکی از چالش‌های بنیادین در طراحی پردازنده‌های خط لوله، یعنی مدیریت مخاطرات کنترلی^{۱۲۵} ناشی از دستورهای انشعاب بپردازیم. در این پروژه باید چندین روش مختلف را برای مدیریت جریان دستورات پیاده‌سازی شده و رفتار آن‌ها با استفاده از پارامترهای اندازه‌گیری یکسان مقایسه می‌شوند. هدف نهایی، درک عمیق موازنه میان سادگی سخت‌افزاری و کارایی زمانی سامانه است.

شرح مسئله و ساختار کلی

در پردازنده‌های خط لوله، چندین دستور به‌طور هم‌زمان در مراحل مختلف حضور دارند. هنگام مواجهه با دستورهای انشعاب، نتیجه واقعی یک دستور انشعاب، یعنی تعیین مسیر صحیح اجرا و آدرس مقصد، معمولاً تا مرحله اجرا^{۱۲۶} مشخص نمی‌شود. این تأخیر در شناسایی مسیر، پردازنده را با مسئله کنترل جریان مواجه می‌کند.

ساده‌ترین راه‌حل برای مدیریت این وضعیت، راهبرد توقف^{۱۲۷} است. در این روش، به‌محض شناسایی هر نوع دستور پرشی، شرطی یا غیرشرطی، در مرحله واکنشی^{۱۲۸}، ورود دستورات جدید به خط لوله متوقف می‌شود و این توقف تا مشخص شدن نتیجه انشعاب در مرحله اجرا ادامه می‌یابد. اگرچه این روش از نظر پیاده‌سازی ساده است و نیازی به پیش‌بینی انشعاب یا عملکرد تخلیه^{۱۲۹} ندارد، اما باعث ایجاد توقف‌های مکرر در خط لوله، افزایش تعداد چرخه‌ها و کاهش محسوس کارایی می‌شود.

اگر پردازنده تا زمان مشخص شدن نتیجه قطعی متوقف شود، جریمه زمانی سنگینی به سامانه تحمیل می‌گردد. به همین دلیل، در پردازنده‌های واقعی از عملکردهای پیش‌بینی استفاده می‌شود تا پردازنده مسیر احتمالی را حدس زده و به اجرای دستورات ادامه دهد. اما اگر این حدس اشتباه باشد، دستوراتی که به‌اشتباه وارد خط لوله شده‌اند باید با عملکرد تخلیه حذف شوند که خود دارای هزینه زمانی است.

هدف اصلی این پروژه بررسی میزان بهبود عملکرد پردازنده با استفاده از روش‌های پیشرفته پیش‌بینی انشعاب نسبت به روش پایه (راهبرد توقف) است. با اجرای معیارها و برنامه‌های آزمایشی یکسان، تأثیر این راهبردها بر شاخص‌های عملکرد پردازنده ارزیابی و میزان بهبود حاصل از آن‌ها تحلیل خواهد شد.

اهداف پروژه

- پیاده‌سازی سخت‌افزاری یک پیش‌بینی‌کننده ایستای جهت‌دار^{۱۳۰}، یعنی BTFNT، و یک پیش‌بینی‌کننده پویای مبتنی بر تاریخچه^{۱۳۱}، یعنی BHT یک‌بیتی،
- طراحی دقیق عملکرد تخلیه خط لوله و مدیریت ثبات‌های میانی در صورت بروز خطا،

¹²⁴Branch Prediction

¹²⁵Control Hazards

¹²⁶Execute

¹²⁷Stall Strategy

¹²⁸Fetch

¹²⁹Flush

¹³⁰Static Direction-Based Predictor

¹³¹Dynamic History-Based Predictor

- تحلیل آماری نرخ نقصان پیش‌بینی و تأثیر آن بر تعداد کل چرخه‌های اجرای برنامه.

راهنماهای مورد انتظار برای پیاده‌سازی

۱. پیش‌بینی ایستا به روش ^{۱۳۲}BTFNT

در این راهنما، تصمیم‌گیری بر اساس جهت پرش انجام می‌شود:

- اگر مقصد انشعاب آدرسی کوچک‌تر از شمارنده برنامه ^{۱۳۳} باشد، یعنی پرش به عقب رخ دهد، انشعاب «گرفته‌شده» پیش‌بینی می‌شود،

- اگر مقصد جلوتر باشد، یعنی پرش به جلو رخ دهد، انشعاب «گرفته‌نشده» پیش‌بینی می‌شود.

در صورت اشتباه بودن این حدس، خط لوله باید در مرحله اجرا تخلیه شده و PC با مقدار صحیح اصلاح گردد.

۲. پیش‌بینی پویا با جدول تاریخچه یک‌بیتی ^{۱۳۴}

در این معماری، یک جدول ۶۴ خانه‌ای (^{۲۶}) طراحی می‌شود که تاریخچه آخرین رفتار هر انشعاب را در یک بیت ذخیره می‌کند.

$$index = PC[5 : 0] \Rightarrow BHT[index]$$

- اندیس‌گذاری: اندیس جدول از ۶ بیت کم‌ارزش آدرس دستور در PC به‌دست می‌آید ($index = PC[5:0]$).

- منطق کار: در مرحله واکنشی، بیت متناظر خوانده شده و پیش‌بینی انجام می‌شود. در مرحله اجرا، پس از مشخص شدن نتیجه واقعی، مقدار همان خانه در جدول به‌روزرسانی می‌شود،

- مقداردهی اولیه: در این پروژه، BHT به‌صورت یک ثبات ۶۴ بیتی در سخت‌افزار پیاده‌سازی شده است که در ابتدای اجرا به‌طور کامل با صفر مقداردهی اولیه می‌شود.

مراحل پیاده‌سازی و نکات فنی

(برای انجام این بخش، می‌توانید پیاده‌سازی تمرین ۷ را مبنا قرار داده و آن را توسعه دهید.)

۱. طراحی عملکرد تخلیه خط لوله

ابتدا باید تعیین کنید کدام ثبات‌های خط لوله، مانند IF/ID و ID/EX، در صورت پیش‌بینی اشتباه باید پاکسازی شوند. تخلیه باید به‌گونه‌ای باشد که دستورات قبل از انشعاب، در مراحل MEM یا WB، تغییر نکنند. این کار با جایگزینی دستورات با NOP یا غیرفعال کردن سیگنال‌های نوشتن انجام می‌شود. هزینه زمانی تخلیه برابر است با تعداد مرحله‌هایی که قبل از مرحله تشخیص قرار دارند و دستور اشتباه در آن‌ها وجود دارد.

^{۱۳۲}Backward Taken, Forward Not Taken

^{۱۳۳}Program Counter - PC

^{۱۳۴}One-Bit Branch History Table - BHT

۲. مدیریت اختصاصی پرش‌های غیرشرطی

از آنجا که روش‌های ۱ و ۲ فقط برای پیش‌بینی انشعاب‌های شرطی طراحی شده‌اند، لازم است برای پرش‌های غیرشرطی در این دو حالت، منطق جداگانه‌ای در نظر گرفته شود. برای افزایش کارایی، پرش‌های غیرشرطی باید در مرحله رمزگشایی^{۱۳۵} شناسایی شده و آدرس مقصد همان‌جا محاسبه شود. با این کار، جریمه زمانی تنها ۱ چرخه خواهد بود و نیازی به اشغال فضای جدول پیش‌بینی نیست.

۳. الزامات کد آزمایشی و ارزیابی

برای مقایسه منصفانه هر سه روش، باید از یک برنامه اسمبلی واحد استفاده کنید که دارای ویژگی‌های زیر باشد:

- حداقل شامل یک حلقه تکرار با بیش از ۱۰ مرتبه اجرا باشد،
- شامل ترکیبی از انشعاب‌های شرطی در داخل و خارج حلقه باشد.

۴. پارامترهای اندازه‌گیری عملکرد

برای هر سه روش، مقادیر زیر را استخراج کنید:

- کل چرخه‌ها: تعداد کل پالس‌های ساعت از شروع تا پایان برنامه،
- تعداد کل انشعاب‌ها: شمارش تمامی دستورات پرش اجراشده،
- پیش‌بینی‌های نادرست^{۱۳۶}: تعداد دفعاتی که پردازنده مسیر اشتباه را حدس زده و مجبور به تخلیه شده است. این عدد برای روش توقف صفر است، زیرا هیچ پیش‌بینی‌ای انجام نمی‌شود،
- نرخ نقصان پیش‌بینی^{۱۳۷} (MPR): درصد پیش‌بینی‌های نادرست نسبت به کل انشعاب‌ها.

خروجی‌های مورد انتظار در گزارش و کدهای تحویلی

گزارش نهایی باید شامل بخش‌های زیر باشد:

- تشریح منطق: توضیح گام‌به‌گام نحوه مقداردهی اولیه، به‌روزرسانی و اعمال تخلیه،
- تحلیل ریاضی: ارائه فرمول محاسبه چرخه‌ها و بررسی سربار ناشی از تخلیه:

$$TotalCycles = BaseCycles + (Mispredictions \times Penalty)$$

- جدول مقایسه‌ای نهایی: ارائه نتایج هر ۳ روش، یعنی توقف، ایستا و پویا، در قالب یک جدول برای پارامترهای ذکرشده،
- تحلیل رفتاری: بررسی تأثیر الگوی برنامه‌نویسی بر دقت پیش‌بینی‌کننده‌ها و مقایسه کارایی روش‌ها.

¹³⁵Decode

¹³⁶Mispredictions

¹³⁷Misprediction Rate

علاوه بر گزارش، کدهای تحویلی و نتایج آزمون شما نیز باید شامل این موارد باشد:

- فایل‌های کد منبع سخت‌افزاری، شامل محیط شبیه‌سازی و پیاده‌سازی واحد پیش‌بینی انشعاب،
- ارائه شبیه‌سازی عملکرد صحیح پردازنده با هر سه روش مدیریت مخاطرات کنترلی،
- ارائه نتایج اجرای برنامه‌های آزمایشی برای تأیید صحت عملکرد پیاده‌سازی.

پروژه ۶: تحلیل نقصان‌های برخورد و طراحی معماری حافظه نهان قربانی^{۱۳۸}

در این پروژه، قصد داریم به یکی از چالش‌های مهم در طراحی حافظه نهان، یعنی افت عملکرد ناشی از نقصان‌های برخورد^{۱۳۹} بپردازیم. این مشکل به‌طور ویژه در حافظه‌های نهانی که با روش نگاشت مستقیم^{۱۴۰} یا انجمنی^{۱۴۱} با درجه پایین کار می‌کنند، رخ می‌دهد. این پروژه در شبیه‌ساز آکیتا پیاده‌سازی و تحلیل می‌شود.

شرح مسئله و ساختار کلی

معمولاً به دلیل محدودیت‌هایی که در ساختارهای نگاشت حافظه وجود دارد، ممکن است چندین آدرس پرکاربرد دقیقاً به یک خط از حافظه نهان اختصاص یابند. حتی اگر ظرفیت کلی حافظه نهان برای نگهداری داده‌ها کافی باشد، تداخل آدرس‌ها باعث می‌شود داده‌ها پیوسته از حافظه اخراج شده و پدیده‌ای به نام تلاطم^{۱۴۲} رخ دهد. این اتفاق، سامانه را مجبور می‌کند تا داده‌ها را بارها از سطوح پایین‌تر حافظه واکشی کند که جریمه زمانی و تأخیر قابل توجهی را به همراه دارد.

در ابتدا این رفتار ناکارآمد را در شبیه‌ساز مدل‌سازی کرده و شمارنده‌هایی برای ثبت نقصان‌های برخورد استفاده کنید. سپس با تغییر معماری و اضافه کردن یک پیمانه جدید به نام «حافظه نهان قربانی»، راه‌حلی برای کاهش ترافیک اضافی حافظه ارائه دهید. در نهایت باید نشان دهید که این راه‌حل چه تأثیری بر موازنه میان هزینه سخت‌افزاری و زمان اجرا دارد.

اهداف پروژه

- درک عملی پدیده‌هایی مانند اخراج مکرر داده‌ها، تلاطم حافظه و تشخیص تفاوت میان نقصان برخورد و نقصان ظرفیتی^{۱۴۳}،
- تحلیل تأثیر میزان انجمنی بودن حافظه بر نرخ اصابت و ترافیک حافظه،
- طراحی و پیاده‌سازی سخت‌افزاری یک پیمانه جدید برای حافظه نهان قربانی و مدیریت فرایند نسبتاً پیچیده جابه‌جایی دوطرفه^{۱۴۴}.

توضیح مفهومی عملکرد حافظه نهان قربانی

در معماری‌های پایه و مرسوم، وقتی یک بلاک جدید به دلیل نقصان عدم اصابت وارد حافظه نهان L1 می‌شود، بلاک قدیمی موجود در آن جایگاه بلافاصله اخراج^{۱۴۵} شده و به L2 فرستاده می‌شود. حال اگر پردازنده دوباره به همان بلاک اخراج‌شده نیاز پیدا کند، باید تأخیر سنگین خواندن دوباره آن از L2 را تحمل کند.

¹³⁸Victim Cache

¹³⁹Conflict Misses

¹⁴⁰Direct-Mapped

¹⁴¹Associative

¹⁴²Thrashing

¹⁴³Capacity Miss

¹⁴⁴Swap

¹⁴⁵Evict

برای حل این مشکل، یک بافر بسیار کوچک تمام‌انجمنی^{۱۴۶} بین L1 و L2 قرار می‌گیرد که به آن حافظه نهان قربانی می‌گوییم. روند کار به این صورت است که بلاک‌های اخراجی از L1 ابتدا وارد این بافر می‌شوند. اگر پردازنده داده‌ای را درخواست کند که در L1 نیست، اما داخل حافظه نهان قربانی وجود دارد، فرایندی به نام جابه‌جایی رخ می‌دهد؛ یعنی داده درخواستی سریعاً به L1 برمی‌گردد و هم‌زمان، بلاک اخراجی جدید از L1 جای آن را در حافظه نهان قربانی می‌گیرد. با این روش، از ارسال درخواست‌های اضافی و پرهزینه به L2 جلوگیری می‌شود.

مراحل پیاده‌سازی و نکات فنی

۱. پیاده‌سازی ساختار پایه

- طراحی یک برنامه محک^{۱۴۷} در شبیه‌ساز آکیتا با هدف ایجاد ترافیک متناوب. فاصله آدرس‌ها^{۱۴۸} در این برنامه باید مضربی از ظرفیت حافظه نهان باشد تا بتواند بیشترین میزان نقصان برخورد را شبیه‌سازی کند.
- اضافه کردن شمارنده‌های سخت‌افزاری برای ثبت تعداد دفعات اصابت/نقصان در L1 و تعداد درخواست‌های خواندن/نوشتن ارسالی به L2.

۲. اندازه‌گیری و مشاهده افت عملکرد

- اجرای برنامه‌های محک روی حافظه نهان نگاشت مستقیم، یعنی ساختار پایه.
- محاسبه میانگین زمان دسترسی به حافظه^{۱۴۹} در حالت پایه.
- نشان دادن میزان ترافیک هدررفته و تأخیر تحمیل‌شده به پردازنده در اثر پدیده تلاطم^{۱۵۰}.

۳. تحلیل معماری حافظه نهان

- بررسی ماشین حالت^{۱۵۱} ارتباطی بین L1 و L2 در معماری پایه،
- تحلیل مسیر رویدادهای خواندن و نوشتن و نحوه مدیریت پیام‌های مربوط به اخراج داده‌ها،
- بررسی چالش بن‌بست^{۱۵۲} در تبادل هم‌زمان پیام‌ها بین عناصر^{۱۵۳}.

۴. طراحی و پیاده‌سازی راه‌حل

- مسیریابی پیام‌ها^{۱۵۴}: تغییر منطق مسیریابی تا بلاک‌های اخراجی از L1 به‌جای رفتن به L2، مستقیماً به حافظه نهان قربانی هدایت شوند،

¹⁴⁶Fully Associative

¹⁴⁷Benchmark

¹⁴⁸Stride

¹⁴⁹Average Memory Access Time - AMAT

¹⁵⁰Thrashing

¹⁵¹State Machine

¹⁵²Deadlock

¹⁵³Components

¹⁵⁴Message Routing

- فرایند جابه‌جایی: مدیریت ناهمگام^{۱۵۵} رویدادها به گونه‌ای که در صورت اصابت در حافظه قربانی، بلاک مورد نیاز به‌طور ایمن به L1 برود و بلاک اخراجی جدید وارد پیمانه قربانی شود،
- سیاست جایگزینی: روشی مانند FIFO یا LRU برای مدیریت سرریز ظرفیت محدود حافظه نهان قربانی، برای مثال اندازه ۸ بلاک، به سمت L2 پیاده‌سازی شود.

۵. مقایسه قبل و بعد از اعمال حافظه نهان قربانی

- اجرای دوباره برنامه‌های محک پس از اعمال تغییرات،
- استخراج زمان کل اجرا^{۱۵۶} و محاسبه نرخ اصابت در حافظه نهان قربانی،
- مقایسه میزان کاهش ترافیک L2 و محاسبه فرمول ارتقایافته AMAT به شکل زیر:

$$AMAT = HitTime_{L1} + MissRate_{L1} \times (HitRate_{VC} \times Penalty_{VC} + MissRate_{VC} \times Penalty_{L2})$$

خروجی‌های مورد انتظار در گزارش و کدهای تحویلی

گزارش نهایی شما باید شامل موارد زیر باشد:

- رسم نمودار ماشین حالت و مسیریابی پیام‌ها در هر دو معماری پایه و دارای حافظه نهان قربانی،
- توضیحات دقیق درباره نحوه مدل‌سازی و مدیریت فرایند هم‌زمان جابه‌جایی بدون بروز بن‌بست در شبیه‌ساز،
- تحلیل ریاضی و فرمولی AMAT در دو حالت قبل و بعد از اضافه شدن پیمانه،
- شرح تغییرات اعمال‌شده در کد شبیه‌ساز و فایل‌های پیکربندی^{۱۵۷} شبکه حافظه‌های نهان،
- نمودارهای مقایسه‌ای که میزان کاهش ترافیک حافظه L2 را نشان دهند.

علاوه بر گزارش، نتایج آزمون شما نیز باید شامل این موارد باشد:

- کدهای پیاده‌سازی‌شده، یا عناصر نوشته‌شده، به زبان Go،
- ارائه مشاهدات عملی از نقصان‌های برخورد در حالت پایه با استفاده از اعداد دقیق به‌دست‌آمده از شمارنده‌ها،
- مقایسه کمی تأخیرهای سامانه و تأثیر آن‌ها بر زمان اجرای نهایی برنامه‌ها.

¹⁵⁵ Asynchronous

¹⁵⁶ Execution Time

¹⁵⁷ Configuration

پروژه ۷: پیاده‌سازی حافظه نهان کج‌شده^{۱۵۸} و تحلیل کاهش نقصان برخورد^{۱۵۹}

شرح مسئله و ساختار کلی

در معماری پردازنده‌های نوین، عملکرد حافظه نهان نقشی حیاتی در کارایی کل سامانه ایفا می‌کند. یکی از چالش‌های بنیادین در طراحی حافظه‌های نهان انجمنی مجموعه‌ای^{۱۶۰}، نقصان برخورد است. این نوع نقصان زمانی رخ می‌دهد که تعداد زیادی از بلاک‌های حافظه که به‌صورت مکرر مورد دسترسی قرار می‌گیرند، به دلیل استفاده از یک تابع نگاشت ساده، به یک مجموعه یکسان در حافظه نهان نگاشت شوند. در چنین شرایطی، حتی اگر ظرفیت کلی حافظه نهان کافی باشد، این بلاک‌ها به‌طور مداوم یکدیگر را از مجموعه مذکور بیرون می‌رانند؛ این پدیده تلاطم^{۱۶۱} نام دارد و باعث افت شدید عملکرد می‌شود.

در یک حافظه نهان انجمنی مجموعه‌ای استاندارد با راه^{۱۶۲}، تمام راه‌ها از یک تابع نگاشت یکسان برای تعیین شماره مجموعه استفاده می‌کنند. این تابع معمولاً به‌صورت زیر تعریف می‌شود:

$$SetIndex = \text{BlockAddress} \bmod N_{sets}$$

این سادگی در سخت‌افزار، هزینه سنگینی در کارایی دارد. آدرس‌هایی که فاصله آن‌ها مضربی از اندازه یک راه حافظه نهان ($N_{sets} \times \text{BlockSize}$) است، همواره با یکدیگر در یک مجموعه رقابت می‌کنند.

معماری حافظه نهان برای حل این مشکل ارائه شده است. ایده اصلی این است که به‌جای یک تابع نگاشت مشترک، برای هر راه (Way_i) از یک تابع درهم‌ساز^{۱۶۳} مستقل و متفاوت (H_i) استفاده شود. بدین ترتیب، دو آدرسی که در Way_0 با یکدیگر برخورد می‌کنند، با احتمال بسیار بالا در Way_1 به مجموعه‌های کاملاً متفاوتی نگاشت می‌شوند و می‌توانند به‌صورت هم‌زمان در حافظه نهان حضور داشته باشند. این تکنیک بدون نیاز به افزایش اندازه یا درجه انجمنی بودن حافظه نهان، نرخ نقصان برخورد را به‌شکل چشمگیری کاهش می‌دهد.

در این پروژه، دانشجو ابتدا با ایجاد یک الگوی دسترسی مخرب در شبیه‌ساز آکیتا، پدیده تلاطم را بازسازی و مشاهده می‌کند. سپس با پیاده‌سازی معماری حافظه نهان و استفاده از توابع درهم‌ساز گوناگون، تأثیر این معماری را در کاهش نقصان برخورد و بهبود توزیع بار میان مجموعه‌ها به‌صورت کمی و عملی تحلیل خواهد کرد.

اهداف پروژه

- درک عمیق نقصان برخورد: تحلیل عملی تفاوت میان نقصان برخورد، نقصان ظرفیتی^{۱۶۴} و نقصان اجباری^{۱۶۵} از طریق پیاده‌سازی شمارنده‌های مجزا در شبیه‌ساز،
- پیاده‌سازی توابع درهم‌ساز: طراحی و پیاده‌سازی حداقل دو تابع درهم‌ساز مستقل و کارا، مانند تاکردن XOR و درهم‌سازی فیبوناچی، در زبان برنامه‌نویسی Go،

¹⁵⁸Skewed Cache

¹⁵⁹Conflict Miss

¹⁶⁰Set-Associative

¹⁶¹Thrashing

¹⁶²Way

¹⁶³Hash Function

¹⁶⁴Capacity Miss

¹⁶⁵Compulsory Miss

- اصلاح معماری حافظه نهان: تغییر منطق جستجو، جای‌گذاری و جایگزینی در کنترل‌کننده حافظه نهان شبیه‌ساز آکیتا برای پشتیبانی از نگاشت کج‌شده،
- تحلیل کمی و مقایسه‌ای: اندازه‌گیری و مقایسه دقیق نرخ نقصان، توزیع بار میان مجموعه‌ها و عملکرد کلی سامانه قبل و بعد از پیاده‌سازی معماری جدید با استفاده از برنامه‌های محک هدفمند،
- ارزیابی موازنه طراحی: تحلیل موازنه^{۱۶۶} میان پیچیدگی سخت‌افزاری توابع درهم‌ساز و میزان بهبود عملکرد به‌دست‌آمده.

مراحل پیاده‌سازی و نکات فنی

برای انجام این پروژه، دانشجو باید مراحل زیر را به‌ترتیب طی کند:

۱. تحلیل و آماده‌سازی حافظه نهان پایه

در این مرحله، باید ساختار حافظه نهان انجمنی مجموعه‌ای استاندارد موجود در شبیه‌ساز آکیتا را درک کرده و ابزارهای لازم برای تحلیل را به آن اضافه کنید.

۱. شناسایی عناصر^{۱۶۷}: فایل‌های مربوط به حافظه نهان، معمولاً در فایلی مانند cache.go، را پیدا کرده و ساختار داده‌ای^{۱۶۸} آن را که شامل تگ‌ها، داده‌ها و بیت‌های وضعیت است، به‌دقت بررسی کنید.

۲. افزودن شمارنده‌ها: شمارنده‌های جدیدی برای ثبت آمار دقیق به ساختار حافظه نهان اضافه کنید:

- totalAccesses: تعداد کل درخواست‌های خواندن و نوشتن،
- totalHits: تعداد کل اصابت‌ها،
- compulsoryMisses: نقصان‌هایی که هنگام دسترسی به یک بلاک برای نخستین بار رخ می‌دهند و برای ردیابی آن‌ها به یک ساختار داده اضافی مانند فیلتر بلوم یا جدول هش نیاز است،
- conflictMisses: نقصان‌هایی که به دلیل پر بودن تمام راه‌های یک مجموعه رخ می‌دهند، در حالی که مجموعه‌های دیگر ظرفیت خالی دارند،
- capacityMisses: نقصان‌هایی که به دلیل پر بودن کل ظرفیت حافظه نهان رخ می‌دهند،
- setUtilization: یک آرایه برای شمارش تعداد دسترسی به هر مجموعه.

۳. اصلاح ماشین حالت^{۱۶۹}: منطق ماشین حالت کنترل‌کننده حافظه نهان را اصلاح کنید تا در هر دسترسی، این شمارنده‌ها را به‌درستی به‌روزرسانی کند.

¹⁶⁶Trade-off

¹⁶⁷Components

¹⁶⁸struct

¹⁶⁹State Machine

۲. بازسازی و مشاهده پدیده تلاطم

هدف این مرحله، ایجاد شرایطی است که در آن نقصان برخورد به وضوح قابل مشاهده باشد.

۱. طراحی برنامه محک^{۱۷۰}: یک برنامه محک ساده بنویسید که به یک آرایه بزرگ با گام^{۱۷۱} مشخص دسترسی پیدا کند. گام را برابر با اندازه یک راه حافظه نهان قرار دهید:

$$\text{stride} = N_{sets} \times \text{BlockSize}$$

این کار باعث می شود تمام دسترسی ها به یک مجموعه یکسان نگاشت شوند.

۲. اجرای شبیه سازی: برنامه محک را روی شبیه ساز با حافظه نهان پایه اجرا کنید.

۳. تحلیل نتایج: خروجی شمارنده ها را بررسی کنید. انتظار می رود:

- نرخ بالای نقصان برخورد.
- توزیع بار بسیار نامتوازن در آرایه `setUtilization` به گونه ای که یک یا چند مجموعه تعداد دسترسی بسیار بالا و بقیه تقریباً صفر داشته باشند.

۳. طراحی و پیاده سازی توابع درهم ساز

در این گام، توابع نگاشت مستقل برای هر راه پیاده سازی می شوند.

۱. ایجاد توابع مستقل: حداقل دو تابع درهم ساز متفاوت را به عنوان توابع کمکی در فایل حافظه نهان پیاده سازی کنید. برای مثال:

- تابع تا کردن **XOR**: این تابع با ترکیب بیت های بالا و پایین آدرس بلاک، وابستگی های ساده را از بین می برد،

$$H_{XOR}(\text{blockAddr}, N_{sets}) = (\text{blockAddr} \oplus (\text{blockAddr} \gg \log_2(N_{sets}))) \bmod N_{sets}$$

- تابع درهم سازی فیبوناچی: این تابع از خواص عدد طلایی برای توزیع یکنواخت کلیدها استفاده می کند و کیفیت بالاتری دارد،

$$H_{Fib}(\text{blockAddr}, N_{sets}) = ((\text{blockAddr} \times 0x9E3779B97F4A7C15) \gg (64 - \log_2(N_{sets})))$$

۲. ایجاد یک تابع توزیع کننده: یک تابع اصلی بنویسید که شماره راه (`wayID`) و آدرس بلاک را به عنوان ورودی دریافت کرده و بر اساس آن، یکی از توابع درهم ساز را فراخوانی کند.

$$\text{mapToSet}(\text{blockAddr}, \text{wayID}) \rightarrow \text{setIndex}$$

¹⁷⁰Benchmark

¹⁷¹Stride

۴. یکپارچه‌سازی منطق کج‌شده در حافظه نهان

این حساس‌ترین مرحله پروژه است که در آن منطق اصلی حافظه نهان تغییر می‌کند.

۱. اصلاح فرایند جستجو^{۱۷۲}:

- به‌جای محاسبه یک شماره مجموعه، باید یک حلقه به تعداد راه‌ها (W) اجرا شود،
- در هر تکرار i ، شماره مجموعه برای Way_i با استفاده از تابع H_i محاسبه می‌شود: $set_i = H_i(\text{blockAddr})$.
- سپس تگ آدرس ورودی با تگ ذخیره‌شده در بلاک مربوطه در set_i و Way_i مقایسه می‌شود.
- در صورت یافتن تگ در هر یک از راه‌ها، اصابت رخ می‌دهد.

۲. اصلاح فرایند جایگزینی^{۱۷۳} در زمان نقصان:

- در صورت بروز نقصان، اکنون W بلاک کاندید برای جایگزینی وجود دارد؛ یکی از هر مجموعه set_i در Way_i .
- سیاست جایگزینی، مانند LRU، باید بین این W بلاک کاندید اجرا شود تا قربانی نهایی را انتخاب کند. این کار نیازمند اصلاح ساختار داده‌های LRU است تا اطلاعات زمانی را برای بلاک‌هایی که در مجموعه‌های مختلف قرار دارند، مدیریت کند.

۵. ارزیابی و تحلیل جامع نتایج

در مرحله نهایی، باید کارایی معماری جدید را با حالت پایه مقایسه کنید.

۱. اجرای مجدد محک‌ها: برنامه محک ایجادکننده تلاطم و همچنین چند برنامه محک استاندارد دیگر، مانند برنامه‌هایی با الگوهای دسترسی محلی یا تصادفی، را روی حافظه نهان اجرا کنید.

۲. مقایسه داده‌ها: نمودارهای مقایسه‌ای دقیق برای موارد زیر تهیه کنید:

- نرخ نقصان: نمودار میله‌ای مقایسه نرخ نقصان کل و نرخ نقصان برخورد بین حالت پایه و حالت برای هر محک،
- توزیع بار: هیستوگرام $setUtilization$ برای هر دو معماری رسم کنید. انتظار می‌رود در حالت، توزیع بسیار یکنواخت‌تر باشد،
- عملکرد کلی: معیارهایی مانند تعداد کل چرخه‌های اجرای برنامه یا دستورالعمل بر چرخه^{۱۷۴} را مقایسه کنید.

۳. نتیجه‌گیری: بر اساس نتایج کمی، تحلیل کنید که معماری حافظه نهان تا چه حد در کاهش نقصان برخورد مؤثر بوده و کدام تابع درهم‌ساز عملکرد بهتری داشته است. همچنین به پیچیدگی سخت‌افزاری اضافه‌شده و تأثیر آن بر تأخیر دسترسی اشاره کنید.

¹⁷²Lookup

¹⁷³Replacement

¹⁷⁴Instructions Per Cycle - IPC

خروجی‌های مورد انتظار در گزارش و کدهای تحویلی

گزارش نهایی شما باید شامل موارد زیر باشد:

- شرح کامل تغییرات اعمال شده در کد آکیتا و توضیح معماری حافظه نهان،
- رسم نمودار ماشین حالت جدید برای فرایندهای جستجو و جایگزینی،
- توضیح نحوه پیاده‌سازی توابع درهم‌ساز و تابع توزیع‌کننده در کنترل‌کننده حافظه نهان،
- ارائه جداول و نمودارهای مقایسه‌ای برای نمایش نرخ نقصان کل، نرخ نقصان برخورد و توزیع بار میان مجموعه‌ها،
- تحلیل علت کاهش توزیع بار بهبود یافته و نقصان برخورد،
- بررسی شرایطی که ممکن است حافظه نهان عملکرد ضعیف‌تری داشته باشد، برای مثال افزایش تأخیر به دلیل محاسبات درهم‌سازی.

علاوه بر گزارش، کدهای تحویلی و نتایج آزمون شما نیز باید شامل این موارد باشد:

- کد پیاده‌سازی شده توابع درهم‌ساز و تابع توزیع‌کننده در شبیه‌ساز،
- کد تغییرات مربوط به منطق جستجو، جای‌گذاری و جایگزینی در حافظه نهان،
- نتایج اجرای برنامه محک ایجادکننده تلاطم و چند برنامه محک دیگر با الگوهای دسترسی متفاوت،
- خروجی شمارنده‌های مربوط به نقصان اجباری، نقصان ظرفیتی، نقصان برخورد، نرخ اصابت و میزان استفاده از مجموعه‌ها،
- نمودارها یا فایل‌های داده‌ای لازم برای مقایسه حالت پایه و حالت دارای حافظه نهان کج‌شده.